Code Explanation

Test DLT Pipeline
The provided code is a unit test suite for a Delta Live Tables (DLT) pipeline, written using PySpark and the unittest framework. It tests the data transformation logic for customer and order data, as well as the aggregate spend calculation.

Overview
The test suite covers three main test cases: customer silver transformation, order silver transformation, and customer aggregate spend logic. It creates sample customer and order data, applies the transformation logic, and verifies the results.

Step 1: Setting Up the Test Environment
This step involves creating a Spark session and adding the necessary dependencies to the Python path. It also defines the test class `TestDLTPipeline` that inherits from `unittest.TestCase`.

```
import unittest
from pyspark.sql import SparkSession
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file_
 ), '..')))
```

Step 2: Creating Sample Data
In this step, sample customer and order data are created along with their respective schemas. The data includes null values and duplicates to test the transformation logic.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

cls.customer_data = [
    ("C001", "John Doe", "john@example.com", "North"),
    ("C002", "Jane Smith", "jane@example.com", "South"),
    ("C003", "Bob Johnson", "bob@example.com", "East"),
    ("C004", "Alice Brown", "alice@example.com", "West"),
    ("C005", None, "mike@example.com", "North"),  # Null value
    ("C001", "John Doe", "john@example.com", "North")  # Duplicate
]

cls.customer_df = cls.spark.createDataFrame(cls.customer_data, custo
mer_schema)
```

Step 3: Creating Temp Views for Testing
The customer and order DataFrames are registered as temp views to facilitate testing.

```
cls.customer_df.createOrReplaceTempView("customer_bronze_view")
cls.order_df.createOrReplaceTempView("order_bronze_view")
```

Step 4: Testing Customer Silver Transformation
This test case applies the customer silver transformation logic, which involves dropping null values and duplicates.

```
customer_silver_df = self.customer_df.na.drop().dropDuplicates()
self.assertEqual(customer_silver_df.count(), 4)
self.assertEqual(customer_silver_df.filter(customer_silver_df.CustId
 == "C001").count(), 1)
```

Step 5: Testing Order Silver Transformation
The order silver transformation logic is tested, which includes dropping null values and duplicates, and calculating the total amount.

```
order_silver_df = self.order_df.na.drop().dropDuplicates()
    .withColumn("TotalAmount", self.order_df["PricePerUnit"] * self.
order_df["Qty"])
self.assertEqual(order_silver_df.count(), 4)
self.assertTrue("TotalAmount" in order_silver_df.columns)
```

Step 6: Testing Customer Aggregate Spend Logic
This test case verifies the aggregate spend calculation logic by joining customer and order data, applying the aggregation logic, and verifying the results.

```
joined_df = self.customer_df.join(
    self.order_df.na.drop().dropDuplicates().withColumn("TotalAmount
",
        self.order_df["PricePerUnit"] * self.order_df["Qty"]),
    "CustId",
    "inner"
).withColumn("IsActive", lit(True))

aggregated_df = joined_df.filter(joined_df["IsActive"] == True)
    .groupBy("Name", "Date")
    .agg(spark_sum("TotalAmount").alias("TotalAmount"))
self.assertEqual(aggregated_df.count(), 4)
```